

자동화 설계 — 실전 공장을 해부하다

PILOT · 영상 자동화 공장 해부 → 내 설계도 · 강사 진행 대본 (정금구)

사용법

한 페이지 = 한 슬라이드. ★ KEY POINT(청록) = 반드시 전달할 한 문장 · 화면(파랑) = 이 슬라이드에 띄울 것 · 이렇게 말하세요(검정) = 실제 멘트 · 강사 팁(노랑) · 참고(회색) · → 넘어가며.

원칙: 코드·JSON 원본은 학생에게 안 띄운다. 전부 그림·말·미니예시. 완성 영상 1개만 원본 재생.

용어 한 줄 (강사용)

PILOT = Pipeline 조깅다 · I/O 정한다 · Layer 나눈다 · Oversight 지킨다 · Trace 남긴다

파이프라인 순서 있는 단계들 · **아티팩트** 단계가 남기는 결과 파일(서류) · **오케스트레이터** 단계를 순서대로 시키는 지휘자 · MCP AI에 도구 쏘는 USB · API 프로그램끼리 쓰는 주문 창구 · **먹등** 두 번 해도 결과 같음

구분	시간	슬라이드
Part 0 — 오프닝	8분	1~3
Part 1 — 공장 견학 + 두 예시	14분	4~6
Part 2 — P 조깅다 (실패 사례 포함)	15분	7~10
Part 3 — I 정한다 (계약 없으면·서류철)	15분	11~14
Part 4 — L 나눈다 (네 층·WHAT/HOW/WHERE)	16분	15~18
개념 상자 — API·MCP + 이식	7분	19~20
Part 5 — O 지킨다 (브레이크 기준)	9분	21~22
Part 6 — T 남긴다 (재시작 시나리오)	11분	23~24
Part 7 — 설계 실습 (발표 없음)	18분	25~27
Part 8 — 마무리	7분	28~30

표지 — "자동화는 코드가 아니라 설계다"

1분

★ KEY POINT

오늘은 코드가 아니라 '설계'를 배운다. 실전 공장을 해부해 내 설계도를 그린다.

화면

표지 — 강의명 + 우측 PILOT 다섯 글자.

이렇게 말하세요

안녕하세요. 오늘은 자동화 '설계' 시간입니다. 5차에서 '자동화가 왜 실패하는가'를 봤다면, 오늘은 한 발 더 나아가 직접 설계해 봅시다. 그것도 교과서 예제가 아니라, 제가 실제로 돌리고 있는 자동화 공장을 통째로 뜯어보면서요. 오늘 끝나면 여러분 손엔 '내 업무 자동화 설계도' 한 장이 남습니다.

→ 넘어가며: "근데 자동화, 진짜 어려운 게 뭡까요?"

자동화의 진짜 어려움은 '설계'다

4분

★ KEY POINT

자동화는 '툴 붙이기'가 아니다. 진짜 어려움은 어떻게 쪼개고 뭘 주고받게 할지 — 설계다.

화면

흔한 착각 ✕ vs 진짜 핵심 ✓ 2단 + "설계하는 눈을 기른다".

이렇게 말하세요

많은 분이 자동화를 이렇게 생각해요 — "좋은 툴 하나 붙이면 알아서 되겠지", "AI가 똑똑하니까 '다 해줘' 한 번에 시키면 되겠지." 그런데 실제로 해보면 앞부분은 그럴듯하게 되다가 대개 세 번째 단계쯤에서 무너집니다. 왜냐면 자동화의 진짜 어려움은 툴이 아니라 '설계'거든요. 일을 어떻게 쪼개고, 단계끼리 뭘 주고받게 할지. 5차에서 제가 뭐라고 했죠? "AI가 못하는 게 아니라 전달이 부족한 것." 오늘은 그 '전달'을 설계도로 그리는 날입니다. 그래서 오늘 배우는 건 툴 사용법이 아니라 '설계하는 눈'이에요.

참고

5차(BATLR)를 들은 전제. 안 들었어도 '큰 일을 쪼개 잇는다'만 알면 충분.

→ 넘어가며: "그래서 오늘 뭘 하나면."

남의 공장을 해부해, 내 설계도를 그린다

3분

★ KEY POINT

실전 공장을 PILOT 5칼로 해부 → 각자 '내 자동화 설계도' 1장. 코드는 안 짤다.

화면

견학 → 해부(PILOT) → 설계 3카드 + "결과물 = 설계도 1장".

이렇게 말하세요

순서는 셋이에요. 첫째, 공장을 통째로 견학합니다 — 실제로 돌아가는 걸 봐요. 둘째, PILOT이라는 다섯 개의 칼로 그 공장을 해부해서 설계 원리를 뽑습니다. 셋째, 그 원리로 여러분 업무 하나를 설계도로 그려요. 오늘 코드는 한 줄도 안 짤니다. 만드는 건 6·7차에서 배운 방식으로 그다음에 하면 돼요. 오늘은 '설계가 먼저'입니다.

강사 팁

칠판에 P·I·L·O·T 세로로 적어두고, 원칙 하나 끝날 때마다 옆에 한 줄씩 채워가면 몰입도가 확 올라간다.

→ 넘어가며: "자, 그 공장부터 통째로 봅시다."

제품 정보 넣으면 광고 영상이 나온다

6분

★ KEY POINT

입력(제품 정보) → 출력(한국어 음성 광고 영상). 결과물부터 보여주고 충격을 남긴다.

화면

입력→출력 한 장 → 완성 광고 영상 1개 재생(30초) ← 오늘 유일한 '원본 그대로'.

이렇게 말하세요

제가 만든 공장을 소개합니다. 한 줄로 — 제품 정보를 넣으면 한국어 음성이 깔린 광고 영상이 나옵니다. 말보다 결과죠, 하나 보시죠. (영상 재생) 자, 방금 이거 제가 편집 프로그램 붙잡고 만든 게 아니에요. 제품 정보만 넣으면 공장이 단계를 밟아 알아서 뽑아낸 겁니다. 광고 하나에 보통 몇 시간씩 걸리던 걸, 이 설계 덕분에 반복해서 찍어냅니다.

강사 팁

영상은 '감탄 포인트'. 제품 가격·상세 수치는 굳이 설명하지 말 것(민감). 흐름과 결과의 임팩트만.

→ 넘어가며: "이게 어떻게 나오는지 — 흐름을 봅시다."

여덟 칸의 흐름

4분

★ KEY POINT

접수→분석→방향→대본→그림→클립→검수→합본. 각 칸은 판단(Claude) 또는 생성(도구).

화면

아이콘 8칸 화살표 흐름도(원본 문서 X, 단순 재작성).

이렇게 말하세요

흐름은 이래요. 제품을 접수하고, 분석하고, 어떤 느낌으로 갈지 방향을 정하고, 대본을 쓰고, 장면 그림을 만들고, 그림을 영상으로 움직이고, 검수·보정하고, 하나로 합칩니다. 여덟 칸이죠. 여기서 하나 봐주세요 — 앞쪽 접수·분석·대본은 '판단하고 글 쓰는' 일이라 Claude가 하고, 그림·영상은 '생성'이라 전용 도구가 합니다. 한 칸에 두 성격을 섞지 않았어요. 복잡해 보이지만, 지금부터 다섯 개의 칼로 해부하면 원리만 남습니다.

→ 넘어가며: "근데 이 큰 공장, 여러분 업무랑 무슨 상관? 그래서 예시를 하나 더."

공장 옆에 — '내 업무'도 같이 본다

4분

★ KEY POINT

큰 공장(예시 A)과 누구나 쓸 작은 예시(예시 B·주간 리포트)를 나란히 놓고 원칙마다 둘 다 확인.

화면

예시 A(영상 공장) / 예시 B(주간 리포트) 2단.

이렇게 말하세요

"영상 공장은 너무 크잖아, 나랑 상관없어" 하실까 봐, 오늘은 작은 예시를 하나 더 데리고 갑니다. '주간 업무 리포트 자동화' — 매주 흩어진 기록을 모아 리포트 초안을 뽑는 거예요. 누구나 당장 만들 만한 크기죠. 앞으로 다섯 원칙을 하나씩 볼 때마다, 큰 공장에서 한 번, 이 작은 리포트에서 한 번, 이렇게 두 번 확인할 거예요. 그러면 '아, 크든 작든 원리는 똑같구나'가 몸에 남습니다.

→ 넘어가며: "첫 번째 칼 — P, 쪼갬다."

한 방에? vs 여러 칸으로

4분

★ KEY POINT

한 번에 다 시키면 어디서 틀어졌는지 모른다. 쪼개면 틀어진 칸만 다시.

화면

"제품 → ??? → 영상" ✕ vs "여러 칸으로 분해" ✓ 2단.

이렇게 말하세요

첫 번째 칼, P — Pipeline, 쪼개다. 제가 만약 AI한테 "제품 정보 줄게, 광고 영상 만들어줘" 한 방에 시키면? 심중팔구 이상한 게 나와요. 더 큰 문제는, 어디서 틀어졌는지를 모른다는 거예요. 대본이 문제인지, 그림이 문제인지 뒤엎겨버리죠. 그래서 쪼갭니다. 쪼개 놓으면 틀어진 칸만 다시 돌리면 돼요. 대본이 별로면 대본 칸만, 그림이 별로면 그림 칸만. 흔히 "AI가 똑똑하니 한 번에" 생각하는데, 큰 일일수록 정반대예요. 똑똑할수록 쪼개서 시켜야 결과가 좋습니다.

→ 넘어가며: "진짜 한 방에 시키면 어떻게 되는지, 실제 사례를 볼까요?"

"다 해줘"라고 하면 생기는 일

4분

★ KEY POINT

한 방에 시키면 원인 추적 불가·전체 재실행·재현 불가 — 자동화가 아니라 도박이 된다.

화면

증상 1·2·3 목록.

이렇게 말하세요

제가 실제로 초반에 한 방에 시켜봤어요. 세 가지 문제가 생기더라고요. 하나, 앞은 그럴듯한데 뒤로 갈수록 대본과 그림이 따로 놀아요 — 근데 어느 단계가 원인인지 알 수가 없어요. 둘, 한 군데만 고치고 싶는데 전체를 처음부터 다시 돌려야 해요. 시간도 돈(생성 비용)도 폭발하죠. 셋, 이게 제일 치명적인데 — 될 때도 있고 안 될 때도 있어요. 재현이 안 됩니다. 자동화의 생명은 '똑같이 나옴'인데 그게 깨지는 거예요. 결론은 하나예요. 큰 일을 한 프롬프트에 욕여 넣지 마라. 쪼개는 순간 자동화가 됩니다.

강사 팁

"여러분도 AI한테 한 번에 시켰다가 이상하게 나온 적 있죠?" 공감 질문으로 경험 소환.

→ 넘어가며: "그럼 어떻게 쪼개야 잘 쪼개ն 걸까요?"

좋은 단계의 세 조건

4분

★ KEY POINT

① 하나의 책임 ② 끝이 분명(결과물이 딱 떨어짐) ③ 다음 칸이 그걸 받는다. 보통 3~7개.

화면

세 조건 카드 + "3~7단계" 감.

이렇게 말하세요

좋은 단계엔 세 가지 조건이 있어요. 하나, 하나의 책임 — 한 칸은 한 일만 합니다. '분석'하면서 '대본'까지 쓰는 칸은 안 만들어요. 둘, 끝이 분명 — 그 칸이 끝나면 딱 떨어지는 결과물이 나와야 해요. 대본 한 편, 그림 여섯 장처럼요. "대충 진행됨" 같은 건 끝이 아니에요. 셋, 다음 칸이 그 결과물을 받아 이어갑니다. 개수 감을 드리면 보통 3~7단계예요. 2개면 덜 쪼갠 거고, 20개면 관리가 더 복잡해져요. 기준은 딱 하나 — '한 칸 = 한 책임'.

→ 넘어가며: "공장이랑 리포트, 둘 다로 쪼개볼게요."

같은 원리, 두 크기로 쪼개기

3분

★ KEY POINT

공장 8칸 / 리포트 5칸 — 크기는 달라도 '한 칸=한 책임, 끝이 분명'은 똑같다.

화면

위: 공장 8칸 / 아래: 리포트 5칸(수집→분류→요약→작성→검토).

이렇게 말하세요

보세요. 위는 영상 공장 여덟 칸, 아래는 주간 리포트 다섯 칸. 리포트는 수집하고, 프로젝트별로 분류하고, 요약하고, 초안 작성하고, 사람이 검토. 크기는 다르지만 규칙은 똑같아요 — 각 칸이 한 가지 일만 하고, 끝나면 결과물이 딱 나오죠. 여러분 업무도 이렇게 다섯 칸 안쪽으로 쪼개질 겁니다. 이따 직접 해볼 거예요.

→ 넘어가며: "쪼갠으면 — 칸과 칸을 뭘로 잇죠?"

단계를 '계약'으로 잇는다

4분

★ KEY POINT

칸 사이는 '말'이 아니라 정해진 양식의 결과물로. 각 칸의 입력→출력을 못 박는다.

화면

손으로 만든 미니 카드: "대본 단계 ▶ 들어가는 것: 의도 / 나오는 것: 컷별 대사".

이렇게 말하세요

두 번째 칼, I — I/O, 정한다. 칸을 쪼갬으면, 이제 칸과 칸 사이를 뭘로 잇느냐가 문제예요. 답은 '정해진 양식의 결과물'입니다. 각 칸이 뭘 받아서(입력) 뭘 내놓는지(출력)를 계약처럼 못 박아요. 예를 들어 대본 칸은 — 들어가는 건 '영상 의도', 나오는 건 '컷별 대사와 장면 묘사'. 이걸 미리 정해두는 거예요. "알아서 잘 넘겨줘"가 아니라, "너는 이걸 받아서 이 모양으로 내놔"라고요.

→ 넘어가며: "이 양식이 없으면 어떻게 되는지 봅시다."

양식이 없으면 중간에 멈춘다

4분

★ KEY POINT

단계 사이 양식이 안 정해지면 다음 칸이 뭘 받을지 몰라 깨진다 — 자동화가 멈추는 대부분의 이유.

화면

계약 없음 ✕ / 계약 있음 ✓ 2단.

이렇게 말하세요

계약이 없으면 이런 일이 생겨요. 대본 칸이 어떤 날은 대사만 주고, 어떤 날은 줄거리만 줘요. 그럼 다음 그림 칸이 "어? 대사가 있어야 하는데?" 하고 깨집니다. 결국 사람이 매번 사이에 끼어서 손봐야 해요 — 그럼 자동화가 아니죠. 반대로 "항상 컷별 대사 + 장면 묘사" 이렇게 양식을 고정해두면, 다음 칸은 믿고 그대로 받아 진행합니다. 사람이 안 꺼도 흐르죠. 실제로 자동화가 중간에 멈추는 대부분의 이유가 바로 이거예요 — 단계 사이 양식이 안 정해져서.

참고

여기가 강의의 지적 핵심. '계약'이라는 단어를 반복해 각인.

→ 넘어가며: "계약이 있으면 좋은 점이 하나 더 — 갈아끼우기."

계약이 있으면 — 갈아끼울 수 있다

4분

★ KEY POINT

계약이 있으면 칸을 따로 고치고 따로 테스트하고, 실행자(사람↔AI)를 바꿔도 된다.

화면

따로따로 가능 / 실행자 교체 2단 + 2차 I/O 회수.

이렇게 말하세요

계약이 있으면 두 가지가 가능해져요. 첫째, 칸을 따로 고치고 따로 테스트할 수 있어요. 앞 칸이 어떻게 만들었는지 몰라도, 결과물 모양만 약속돼 있으면 되니까요. 둘째, 실행자를 바꿀 수 있어요. 대본을 사람이 쓰든 AI가 쓰든, "의도 받아서 대사 낸다"는 계약만 지키면 뒤 칸은 신경도 안 써요. 이거, 2차 강의에서 배운 입출력 계약 — 포·필·범·메 기억나시죠? 그게 지침 하나에 적용됐다면, 자동화에선 '단계 하나하나의 계약'으로 그대로 올라오는 거예요.

→ 넘어가며: "이 결과물들, 비유하면 '서류철'이에요."

단계 사이 결과물 = 서류철

3분

★ KEY POINT

각 칸이 결과를 '서류(파일=아티팩트)'로 만들어 다음 칸 책상에 올려둔다. 서류만 있으면 누가 이어받아도 된다.

화면

분석 서류 → 방향 서류 → 대본 서류 → 그림 서류 (단계 카드).

이렇게 말하세요

이 결과물들을 비유하면 '서류철'이에요. 분석 칸은 '제품 요약 서류'를 만들어 다음 책상에 올려두고, 방향 칸은 '방향 서류'를, 대본 칸은 '대본 서류'를 올려둡니다. 이런 서류를 전문 용어로 '아티팩트'라고 불러요. 어려운 말 아니고, 그냥 단계가 남기는 결과 파일이에요. 핵심은 — 서류만 제대로 있으면 누가 이어받아도 일이 굴러간다는 거예요. 담당자가 바뀌어도, 심지어 사람이 시로 바뀌어도요.

→ 넘어가며: "자, 이번엔 가로가 아니라 세로로 나눠봅시다."

가로로 쪼개고, 세로로 나눈다

5분

★ KEY POINT

가로(P)=시간 순서 단계 / 세로(L)=성격 분리. 지식·스펙·지휘자·창구 4층.

화면

오늘의 핵심 그림 — 가로축(파이프라인) × 세로축(레이어 4층) 교차도.

이렇게 말하세요

세 번째 칼, L — Layer, 나눈다. 지금까지 P에서 가로로 쪼갬. 이번엔 세로로 나눕니다. 이 그림이 오늘 제일 중요해요. 가로로 쪼개고(단계), 세로로 나눈다(레이어). 세로 네 층은 — 지식(안 변하는 규칙·자료), 스펙(각 칸의 계약서), 지휘자(칸들을 순서대로 시키는 놈), 창구(사람이 요청 넣는 곳). 가로는 '시간'으로 나눈 거고, 세로는 '성격'으로 나눈 거예요. 6차에서 '파일 하나 = 한 책임' 배웠죠? 그게 시스템 규모로 커지면 이렇게 층이 됩니다.

강사 팁

손으로 가로선·세로선을 직접 그으며 "가로, 세로" 반복. 이 교차 그림 하나만 각인시켜도 성공.

→ 넘어가며: "네 층이 각각 뭘 말하는지 하나씩 볼게요."

각 층은 무엇을 맡나

4분

★ KEY POINT

지식(무엇을 아는가)·스펙(계약서)·지휘자(순서·상태)·창구(입출력). 규칙 바꾸면 지식층만, 순서 바꾸면 지휘자층만.

화면

지식/스펙/지휘자/창구 각 한 줄 설명 목록.

이렇게 말하세요

네 층을 하나씩 볼게요. 지식층은 안 변하는 재료·규칙이에요 — 제품 지식, 제작 지침, 스타일 템플릿. "무엇을 아는가"만 담고, 실행은 몰라요. 스펙층은 각 칸의 계약서예요 — 입력·출력·규칙. 누가 실행해도 똑같이 읽히죠. 지휘자층은 칸들을 순서대로 시키고 상태를 관리해요. 창이나 생성은 안 하고 '순서·상태·호출'만. 창구층은 사람이 요청 넣고 결과 받는 곳이고요. 이렇게 나눠두면 뭐가 좋냐 — 규칙 하나 바꾸려면 지식층만 건드리면 되고, 순서 바꾸려면 지휘자층만 건드리면 돼요. 서로 안 엉켜요.

→ 넘어가며: "그래서 지휘자를 얇게 두면 — 통째로 갈아끼울 수 있어요."

얇은 지휘자 — 갈아끼운다

4분

★ KEY POINT

지휘자는 '순서·상태·호출'만. 얇아서, 지휘자만 갈아끼워도 공장이 돈다(Cowork→Python).

화면

오늘 Cowork / 내일 Python+API, "지식·계약은 그대로, 지휘자만 교체".

이렇게 말하세요

여기서 핵심은 지휘자를 '얇게' 두는 거예요. 지휘자는 순서 정하고, 상태 챙기고, 각 칸을 부르기만 해요. 창의도 생성도 직접 안 합니다. 왜 얇게 두냐고요? 지휘자만 갈아끼워도 공장이 돌게 하려고요. 지금 이 공장의 지휘자는 Cowork예요 — 제가 세션에서 직접 켜서 돌리죠. 근데 내일은 지휘자를 파이썬 프로그램으로 바꿔서 사람 없이 밤새 돌릴 수 있어요. 지식이랑 계약은 그대로 두고, 지휘자만 교체하는 거예요. 이게 잘 나눈 설계의 진짜 힘입니다.

→ 넘어가며: "왜 나누면 강한지, 한 문장으로 정리하면 — WHAT·HOW·WHERE."

무엇을 · 어떻게 · 어디서 를 가르친다

3분

★ KEY POINT

지식=무엇을(WHAT), 지휘자=어떻게(HOW), 창구=어디서(WHERE). 가르면 한 곳만 고쳐도 돼서 안 무너진다.

화면

WHAT·HOW·WHERE 카드 3장.

이렇게 말하세요

네 층을 더 쉽게 외우는 법 — 무엇을, 어떻게, 어디서로 가르친다고 생각하세요. 무엇을 아는가는 지식, 어떻게 시키는 가는 지휘자, 어디서 받는가는 창구. 이게 섞여 있으면 하나 바꾸려다 전부 건드려야 해요. 반대로 가르면, 한 곳만 고쳐도 나머지는 그대로예요. 그래서 큰 자동화가 안 무너지는 거예요. 좋은 설계는 결국 '고칠 때 한 곳만 건드리게' 만 드는 거예요.

→ 넘어가며: "잠깐 — 지휘자가 도구를 부른다고 했죠? 그게 API·MCP예요."

MCP = 도구 꽂는 USB · API = 주문 창구

4분

★ KEY POINT

MCP는 AI에 도구 꽂는 USB(지금 방식), API는 프로그램 주문 창구. 딱 이 두 줄.

화면

MCP / API 2단 + (선택) 실제 MCP로 이미지 1장 생성.

이렇게 말하세요

방금 지휘자가 그림·영상 도구를 부른다고 했죠. 그 '도구를 어떻게 부르느냐'가 요즘 많이 듣는 API랑 MCP예요. 딱 두 줄만 기억하세요. MCP는 AI에 도구를 꽂는 USB예요. 붙이면 Claude가 그 도구를 바로 씁니다 — 지금 제 공장이 이 방식이에요, 그림·영상 도구를 MCP로 꽂아줬죠. API는 프로그램끼리 쓰는 주문 창구고요 — 정해진 양식으로 주문하면 정해진 형식으로 답이 옵니다, 식당 주문서처럼요.

강사 팁

붙여둔 MCP로 이미지 1장만 실제 생성해 "방금 이게 MCP예요" 시연(코드 X, 결과만). SDK·REST·폴링은 '이런 말도 있구나'로만.

→ 넘어가며: "그래서 오늘 MCP, 내일 API — 근데 일은 똑같아요."

MCP now → API later, 일은 그대로

3분

★ KEY POINT

판단·생성·승인 — 하는 일은 그대로, 오늘 MCP로 / 내일 API로 부를 뿐. 설계는 안 바뀐다.

화면

대응표: 하는 일 / 오늘(MCP) / 내일(API).

이렇게 말하세요

표로 보면 명확해요. 판단·글쓰기는 오늘은 Cowork 안의 Claude가, 내일은 Claude API가 합니다. 그림·영상 생성은 오늘 MCP로, 내일 API로. 사람 승인은 오늘 채팅으로, 내일 웹 화면으로. 왼쪽 '하는 일'은 하나도 안 바뀌어요. 오른쪽 '부르는 방식'만 바뀌죠. 이게 가능한 이유가 뭐였죠? 지휘자를 얹게 됐기 때문이에요. 설계가 탄탄하면 손발(도구)은 언제든지 갈아끼울 수 있습니다.

→ 넘어가며: "네 번째 칼 — 사람 관문."

비싼 길목엔 사람 관문

5분

★ KEY POINT

자동화라고 다 무인이 아니다. 되돌리기 어렵거나 비싼 단계 앞에 검수 관문(특히 영상 직전).

화면

신호등 4개: 대본→스토리보드→영상→최종.

이렇게 말하세요

네 번째 칼, O — Oversight, 지킨다. 자동화라고 다 무인일 것 같죠? 아니에요. 비싸거나 되돌리기 어려운 지점 앞엔 사람 관문을 겁니다. 제 공장엔 네 곳 있어요. 대본 검수 — 대본이 산으로 가면 뒤가 다 헛수고니까 여기서 잡고. 스토리보드 검수 — 이게 제일 중요해요. 영상 만들기 직전이거든요. 그림이 이상한데 영상까지 만들면 돈이랑 시간이 통째로 날아가요. 여기서 막으면 가장 큰 낭비를 막습니다. 그다음 영상 검수, 최종 승인.

→ 넘어가며: "그럼 관문을 '어디에' 달지는 어떻게 정할까요?"

"어디에 브레이크를 달까"가 설계

4분

★ KEY POINT

비싸거나 되돌리기 어려운 단계 앞에만. 자동 점검→사람 승인→반려 시 직전 칸으로 되돌림.

화면

다는 기준 / 관문 작동(반려 루프) 2단.

이렇게 말하세요

관문을 어디에 달지가 사실 설계의 핵심이에요. 기준은 두 개예요. 그 뒤가 비싼가? (영상 생성처럼 돈·시간이 크게 드나) 그리고 틀리면 되돌리기 어려운가? (한번 발행·발송하면 끝인가) 둘 다 '예'면 그 앞에 관문을 겁니다. 작동은 이래요 — 자동으로 체크리스트 점검하고, 사람이 승인하고, 아니면 직전 칸으로 되돌려요. 이것을 반려 루프라고 해요. 관문이 하나도 없으면 이상한 게 끝까지 가버리고, 너무 많으면 사람이 계속 붙어 있어야 하니 자동화가 아니죠. 비싼 길목에만 딱딱 답니다.

강사 팁

"여러분 업무에서 '한번 나가면 되돌리기 힘든' 지점이 어디예요?" 질문 던져 O칸 예열.

→ 넘어가며: **"마지막 칼 — 그리고 여기서 다 모입니다."**

파일이 곧 상태다

5분

★ KEY POINT

상태를 사람 머리·메모리가 아니라 전부 파일로. 실행 1건 = 폴더 하나. "메모리에 숨은 상태 금지."

화면

runs/날짜_제품/ 폴더 설명 + "MCP now → API later".

이렇게 말하세요

마지막 칼, T — Trace, 남긴다. 자동화가 '어디까지 했나'라는 상태를 어디에 둘까요? 사람 머릿속? 프로그램 메모리? 아니요, 전부 파일로 남깁니다. 제 공장은 영상 하나 만들 때마다 폴더를 하나 만들고, 중간 결과물이라 미디어를 전부 거기 쌓아요. 그래서 세션이 끊겨도 그 폴더만 있으면 다음 칸부터 이어서 돌아가고, 이미 만든 건 건너뛰니까 다시 눌러도 안전해요. 원칙 하나로 외우세요 — "메모리에 숨은 상태 금지, 상태는 전부 파일에."

→ 넘어가며: "이게 왜 강력한지, 실제 상황으로 볼게요."

끊겨도 이어서 돈다

6분

★ KEY POINT

상태가 파일에 있으면 멈춰도 다음 칸부터 이어가고, 지휘자를 바꿔도 같은 폴더로 이어감. PILOT가 여기서 하나로.

화면

타임라인: ① 다시 켜다 → ② 그림부터 이어감 → ③ 지휘자 바뀌도 동일.

이렇게 말하세요

상황을 하나 그려볼게요. 대본까지 만들었는데 노트북이 꺼졌어요. 처음부터 다시? 아니에요. 파일에 상태가 있으니까 — 다시 켜서 폴더를 열면 '아, 대본 서류까지 있네' 하고 어디까지 됐는지 바로 보여요. 그럼 그림 단계부터 이어가면 됩니다. 이미 있는 접수·분석·대본은 건너뛰어요, 이걸 '먹등'이라고 해요, 두 번 해도 결과가 같다는 뜻. 더 놀라운 건, Cowork로 시작했어도 나중에 파이썬 지휘자가 같은 폴더를 읽어서 이어갈 수 있다는 거예요. 자, 여기서 다섯 원칙이 하나로 모여요. 쪼개고(P), 계약으로 잇고(I), 층으로 나누고(L), 관문 두고(O), 상태를 남기면(T) — 멈춰도 이어지고, 지휘자를 갈아끼워도 굴러가는 진짜 자동화가 됩니다.

참고

5차의 '기록(Record)'이 곧 이 T. 시리즈 연결 한 번 짚기.

→ 넘어가며: "자, 이제 여러분 차례입니다."

내 자동화 설계도 그리기

5분 (안내)

★ KEY POINT

PILOT 5칸에 내 업무 하나를. 완성 못 해도 P·I만 채우면 절반은 성공.

화면

PILOT 5칸 질문표 + 설계도 유인물(docx/출력본) 배포.

이렇게 말하세요

자, 이제 여러분 차례예요. 나눠드린 설계도 종이에, 여러분 업무 하나를 PILOT 다섯 칸으로 그려봅니다. 주제 예시 — 콘텐츠 발행, 주간 리포트, 이메일 분류, 데이터 정리, 고객문의 1차 응대. 하나 고르세요. 각 칸의 질문은 이래요. P: 내 업무를 3~7단계로? I: 각 단계 뭐 받아 뭐 내나? L: 안 변하는 것과 시키는 것은? O: 사람이 꼭 확인할 지점은? T: 뭘 파일로 남기나? 완성 못 해도 괜찮아요. P랑 I만 채워도 절반은 성공이에요.

강사 팁

주제 못 정하는 사람에겐 '주간 리포트'를 그대로 따라 그리게 권하기. 유인물 미리 인쇄 배포.

→ 넘어가며: "막막하면 예시 하나를 같이 볼게요."

예시 — "주간 리포트 자동화"

5분 (작성 시작)

★ KEY POINT

수집→분류→요약→작성→검토. 작성 뒤 사람 확인, 작업 폴더에 단계별 파일.

화면

주간 리포트 5칸 완성 예시(파이프라인 + I / L·O·T).

이렇게 말하세요

예시로 주간 리포트를 5칸으로 채워볼게요. P는 수집→분류→요약→작성→검토, 다섯 칸. I는 수집이 '기간'을 받아 '기록 원본'을 내고, 요약이 '묶음'을 받아 '3줄 요약'을 내고, 작성이 '요약'을 받아 '초안'을 내죠. L은 지식이 리포트 양식, 실행이 수집·요약, 창구가 "이번 주 리포트 만들어줘" 한 줄. O는 작성 뒤에 사람이 한 번 확인하고 최종 승인. T는 '날짜_주제' 폴더에 단계별로 저장. 보이시죠? 딱 이 정도면 설계도가 서요. 자, 이제 여러분 것 시작하세요.

→ 넘어가며: "작성하시는 동안 돌아다니며 봐드릴게요."

자주 막히는 지점 — 이렇게 뚫는다

8분 (작성 + 순회)

★ KEY POINT

막힘 4종을 미리 안내 → 개별 작성 + 강사 순회 첨삭. 발표 없이, 각자 완성에 집중.

화면

막힘 1~4 자가 점검 목록.

이렇게 말하세요

(순회하며 안내) 자주 막히는 데를 미리 알려드릴게요. 하나, "단계를 몇 개로?" → 3~7개, 한 칸이 두 가지 일 하면 쪼개세요. 둘, "입력·출력이 헛갈려요" → 앞 칸의 출력이 내 칸의 입력이 맞는지만 확인. 셋, "관문을 어디에?" → 되돌리기 어렵거나 비싼 단계 앞, 딱 한두 곳. 넷, "다 못 채웠어요" → 괜찮아요, P랑 I만 있으면 뼈대는 완성이에요. (돌아다니며 한 명씩 짧게 첨삭) 설계도가 서면 절반은 끝난 거예요. 만드는 건 6·7차에서 배운 Claude Code로 이어가면 됩니다.

강사 팁

발표는 생략. 대신 순회하며 "P칸 좋네요", "여기 O 관문 하나 더" 식 1:1 짧은 피드백으로 각자 완성을 높인다. 시간 남으면 잘 된 것 1개만 익명으로 화면에 띄워 코멘트.

→ 넘어가며: "오늘 배운 걸 한 장으로 정리하고 마칩니다."

PILOT 한 장 요약

3분

★ KEY POINT

P 쪼갬다·I 정한다·L 나눈다·O 지킨다·T 남긴다. 뼈대 P·L, 관절 I, 브레이크 O, 피 T.

화면

PILOT 5줄 요약표.

이렇게 말하세요

정리합니다. PILOT — Pipeline 쪼갬다, I/O 정한다, Layer 나눈다, Oversight 지킨다, Trace 남긴다. 외우는 법은 이래요. 가로로 쪼개고 세로로 나눈다가 뼈대(P·L), 그 뼈를 잇는 관절이 계약(I), 위험한 곳의 브레이크가 관문(O), 그리고 온몸에 흐르는 피가 상태(T)예요. 이 다섯이 서면, 손발인 도구나 지휘자는 언제나 갈아끼우면 됩니다.

→ 넘어가며: "그리고 오늘의 한 문장."

자동화는 코드가 아니라 설계다

2분

★ KEY POINT

"잘 쪼개고 잘 이으면, 지휘자는 갈아끼우면 된다." 여러분 설계도가 그 시작.

화면

관통 문장 크게 + 시리즈 지도(1·2·5·6·7·8차)에서 8차 강조.

이렇게 말하세요

오늘의 한 문장 — 자동화는 코드가 아니라 설계다. 잘 쪼개고 잘 이으면, 지휘자는 갈아끼우면 된다. 여러분 손에 든 설계도가 그 시작이에요. 돌아보면 1차에서 프롬프트, 2차에서 지침과 계약, 5차에서 자동화의 큰 그림, 6·7차에서 직접 만들기를 배웠고, 오늘 8차에서 그걸 전부 '설계'로 묶었어요. 이제 여러분은 툴을 배운 게 아니라, 설계하는 눈을 얻은 겁니다.

→ 넘어가며: "질문 받겠습니다."

고맙습니다 · Q&A

2분

★ KEY POINT

해부해서 원리를 얻었다 — 이제 어떤 자동화든 PILOT로 설계할 수 있다.

이렇게 말하세요

오늘 실전 공장을 해부해서 설계 원리 다섯 개를 얻었습니다. 핵심은 '자동화는 설계다'예요. 이 다섯 칼을 알면 여러분 업무 어떤 것이든 자동화 설계도를 그릴 수 있어요. 질문 받을게요. 고맙습니다.

강사 팁

돌발 질문 대비는 아래 부록(자문자답) 참고. "다음엔 이 설계도를 진짜 프로그램으로 옮겨보자"로 예고.

→ 끝.

이런 질문이 나오면

Q. 이거 우리도 만들 수 있나요?

설계는 오늘 다 배웠어요. 만드는 건 Claude Code로, 6·7차 방식. 규모가 클 뿐 원리는 같습니다.

Q. MCP랑 API 차이 다시?

MCP는 AI에 도구 꽂는 USB, API는 프로그램 주문 창구. MCP 안에도 결국 API가 있어요.

Q. 관문(사람 확인) 없이 완전 자동은 안 되나요?

됩니다. 단 초기엔 관문 두는 걸 권해요. 믿을 만해지면 하나씩 뺍니다.

Q. 단계는 몇 개가 정답?

3~7개. 정답은 없고, '한 칸=한 책임·끝이 분명'이면 됩니다.

Q. 우리 회사엔 영상 같은 거 없는데요?

PILOT은 영상 전용이 아니에요. 리포트·이메일·데이터 정리 등 '여러 단계로 이뤄진 반복 업무'면 다 적용됩니다.